1.1. Системы контроля версий (VCS)

Система контроля версий — это инструмент для отслеживания изменений в файлах (обычно исходном коде), позволяющий возвращаться к предыдущим состояниям, работать в команде и управлять разными версиями продукта.

Типы:

- Локальные (RCS) — только локальное хранение истории.

- Централизованные (CVS, Subversion) — единый сервер с историей.

- Распределённые (Git, Mercurial) — каждый разработчик имеет полную копию репозитория.

Основные операции в Git:

- `git init` — создать репозиторий.

- `git add` — добавить файлы в отслеживание.

- `git commit` — сохранить снимок состояния.

- `git push` / `pull` — синхронизация с удалённым репозиторием.

- `git branch` / `merge` — работа с ветками.

Преимущества: параллельная работа, откат ошибок, аудит изменений.

1.9. Алгоритмы обработки числовых данных

Определение: Методы решения математических задач: от поиска суммы и среднего значения до работы со сложными формулами через встроенные модули.

- Ключевые особенности:
  - Для базовых задач используются функции sum(), min(), max(), abs(), round().
  - Для сложных вычислений (тригонометрия, логарифмы) используется модуль math.
  - Python автоматически поддерживает "длинную арифметику" (целые числа int ограничены только объемом оперативной памяти)

1.10. Коллекции (комплексные типы данных)

Тип	Характеристика	Пример
`list`	изменяемый, упорядоченный	`[1, 2, 3]`
`tuple`	неизменяемый, упорядоченный	`(1, 2, 3)`
`set`	изменяемый, неупорядоченный, уникал.	`{1, 2, 3}`
`frozenset`	неизменяемый set	`frozenset([1,2,3])`
`dict`	пары ключ-значение	`{'a': 1, 'b': 2}`	ключ должен быть неизменяемым

1.14. Алгоритмы сортировки

* Пузырьковая (Bubble): Сравниваем соседей, самый большой «всплывает» в конец. Повторяем, пока всё не встанет на места. (Медленно: $O(n^2)$).

* Вставками (Insertion): Берем элемент и «втыкаем» его в правильное место в уже отсортированной левой части. Как сортировка карт в руке. ($O(n^2)$).

* Быстрая (Quick): Выбираем опорный элемент (pivot), делим массив на «меньше» и «больше» него. Рекурсивно повторяем для обеих половин. ($O(n \log n)$).

* Слиянием (Merge): Делим массив пополам до единичных элементов, а затем сливаем их обратно парами, сразу выстраивая по порядку. ($O(n \log n)$).

* Пирамидальная (Heap): Строим из массива двоичную кучу (дерево), где корень — максимум. Забираем корень, перестраиваем кучу, повторяем. ($O(n \log n)$).

В Python встроенная `sorted()` — Timsort (гибрид).

1.11. Библиотека numpy. Функционал и возможности

NumPy — работа с многомерными массивами и математическими функциями.

Работает в десятки раз быстрее обычных списков Python благодаря реализации на языке C.

Поддерживает векторизацию: операции выполняются сразу над всем массивом

- `ndarray` — основной тип данных (векторные операции).

1. Создание

* np.array([1, 2, 3]) — из списка.

* np.zeros((n, m)) / np.ones() — массив из 0 или 1.

* np.arange(start, stop, step) — массив-диапазон.

* np.linspace(0, 1, 10) — 10 точек от 0 до 1.

2. Форма (Shape)

* .reshape(n, m) — меняет размер (напр. из 1D в 2D).

* .shape — показывает размеры (строки, столбцы).

* .T — переворачивает матрицу (транспонирование).

3. Математика (без циклов!)

* a + b, a * b — действия над каждым элементом.

* np.dot(a, b) — умножение матриц.

* np.sum(), np.mean(), np.std() — сумма, среднее, отклонение.

* np.min(), np.max() — поиск экстремумов.

4. Выбор данных

* a[1:3, :] — срезы (строки с 1 по 2, все столбцы).

* a[a > 0] — фильтр (выбрать только положительные).

1.12. Библиотека pandas. Функционал и возможности

Pandas — для анализа и обработки табличных данных.

- `Series` — одномерный размеченный массив.

- `DataFrame` — двумерная таблица.

1. Загрузка и создание

* pd.read_csv() / pd.read_excel() — чтение данных из файлов.

* pd.DataFrame(data) — создание таблицы из словаря или массива.

2. Просмотр данных

* .head(n) — показать первые n строк.

* .info() — типы данных, пропуски и память.

* .shape — количество строк и столбцов.

3. Выбор и фильтрация

* df['col'] — выбрать один столбец.

* df[['c1', 'c2']] — выбрать несколько столбцов.

* df.loc[index, 'col'] — доступ по имени.

* df.iloc[0:5, 0:2] — доступ по номеру строки/столбца (срезы).

* df[df['age'] > 18] — фильтрация по условию.

4. Изменение и чистка

* .drop('col', axis=1) — удалить столбец.

* .fillna(value) — заполнить пустые ячейки (NaN).

* .dropna() — удалить строки с пустотами.

* .rename(columns={...}) — переименовать столбцы.

5. Агрегация

* .groupby('col').sum() — группировка данных.

* .sort_values('col') — сортировка по столбцу.

* .value_counts() — подсчет уникальных значений.

1.13. Алгоритмы поиска

Линейный поиск — O(n), проверка каждого элемента.

```python
for i in range(len(arr)):
    if arr[i] == target: return i
```

Бинарный поиск — O(log n), только для отсортированных массивов.

```python
def binary_search(arr, x):
```

```
    lo, hi = 0, len(arr)-1
    while lo <= hi:
        mid = (lo+hi)//2
        if arr[mid] == x: return mid
        elif arr[mid] < x: lo = mid+1
        else: hi = mid-1
    return -1
```

1.15. Файлы и файловые операции

Режимы открытия: `r` (чтение), `w` (запись, перезапись), `a` (добавление), `b` (бинарный), `+` (чтение/запись).

Основные операции:

```
f = open("file.txt", "r")
content = f.read()       # всё
line = f.readline()      # одна строка
lines = f.readlines()    # список строк
f.close()
# with-контекстменеджер (рекомендуется)
with open("file.txt", "w") as f:
    f.write("Hello")
# работа с бинарными файлами
with open("image.jpg", "rb") as f:
    data = f.read()
```

1.17. Область действия имён (Scopes)

Область действия имён — это правила, по которым интерпретатор Python ищет значение переменной. В Python действует иерархия **LEGB**:

1. **Local (Локальная)**: Переменные, объявленные внутри функции. Они создаются при вызове функции и удаляются после её завершения.
2. **Enclosing (Замыкающая)**: Актуальна для вложенных функций. Это область внешней (родительской) функции. Для изменения таких переменных используется ключевое слово nonlocal.
3. **Global (Глобальная)**: Имена, определённые на уровне всего файла (.py). Чтобы изменить глобальную переменную внутри функции, нужно объявить её через global.
4. **Built-in (Встроенная)**: Самый верхний уровень. Содержит зарезервированные имена и функции (print, len, int, list и т.д.).

Важный нюанс: В Python нет блочной области видимости (переменные внутри if, for, while не являются локальными).

1.17. Перегрузка функций

В Python нет перегрузки функций в классическом виде (как в C++/Java), потому что имена не привязаны к типам.

2. Аргументы по умолчанию: def func(a, b=None) — позволяет вызывать функцию с разным количеством параметров.

3. Переменные аргументы: *args и **kwargs — функция принимает любое число позиционных и именованных данных.

4. Проверка типов: внутри функции через isinstance(что проверяем, тип) меняем логику в зависимости от того, что пришло (число, строка или список).

5. Декоратор @singledispatch: официальный способ из functools для создания разных сценариев работы функции под разные типы данных.

1.18. Стандартные функции

В Python есть встроенные функции (без импорта):

- `print()`, `input()`, `len()`, `type()`, `id()`
- `int()`, `float()`, `str()`, `bool()`, `list()`, `dict()`
- `range()`, `enumerate()`, `zip()`, `map()`, `filter()`, `sorted()`
- `max()`, `min()`, `sum()`, `abs()`, `round()`
- `open()`, `help()`, `isinstance()`

1.19. Функции с переменным числом параметров

```
def sum_all(*args):        # кортеж позиционных
    return sum(args)
def print_info(**kwargs):  # словарь именованных
    for k, v in kwargs.items():
        print(k, v)
def combined(a, b, *args, opt=10, **kwargs):
    pass
# вызов
sum_all(1,2,3,4)
print_info(name="Alex", age=25)
```

1.20. Рекурсия и рекурсивные функции

Рекурсия — функция вызывает саму себя. Обязательно условие выхода.

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n-1)
```

Фибоначчи

```
def fib(n):
    return n if n <= 1 else fib(n-1) + fib(n-2)
```

Недостатки: глубокий вызов → переполнение стека (`RecursionError`).

1.21. Функции для обработки строк. Определение строк. Преобразование строк и чисел

Определение строк:

```
s1 = 'hello'
s2 = "world"
s3 = """многострочная
строка"""
```

Основные методы строк:

- `upper()`, `lower()`, `strip()`, `split()`, `join()`,` find()`
- `startswith()`, `endswith()`, `isalpha()`, `isdigit()`

Преобразование строка ↔ число:

```
s = "123"
n = int(s)          # строка в int
f = float("3.14")   # в float
str(456)            # число в строку
```

1.22. Лямбда-функции

Лямбда — анонимная функция в одно выражение.

Синтаксис: `lambda параметры: выражение`

```
square = lambda x: x**2
print(square(5))   # 25
```

Ограничения: нет операторов (`if` тернарный можно, но не `while`), только одно выражение.

1.24. Многофайловые приложения

Когда код становится большим, его делят на части для удобства поддержки и повторного использования:

- **Модуль**: Любой файл с расширением .py.
- **Пакет**: Папка, содержащая модули и специальный файл __init__.py (в новых версиях Python он может быть пустым или даже отсутствовать, но для совместимости лучше оставлять).

Импорт позволяет использовать код из других файлов: import module или from module import function. При импорте код модуля выполняется один раз и кэшируется.

Пример структуры проекта

```
my_project/
├── main.py
├── package/
│   ├── __init__.py
│   ├── module1.py
│   └── module2.py
└── utils/
    └── helpers.py
```

sys.path — список путей, где Python ищет модули.

1.25. Классы. Определение, атрибуты, методы

Класс — это пользовательский тип данных, шаблон для создания объектов.

- **Атрибуты**: Переменные, которые хранят состояние (например, color, speed). Бывают **атрибутами класса** (общие для всех экземпляров) и **атрибутами экземпляра** (у каждого свои).
- **Методы**: Функции, определяющие поведение (например, drive()).
- **Параметр self**: Ссылка на конкретный экземпляр класса. Обязателен первым аргументом в методах экземпляра, чтобы метод знал, с чьими данными он работает.

```
class Car:
    wheels = 4                  # атрибут класса (общий)
    def __init__(self, model, color):
        self.model = model      # атрибут экземпляра
        self.color = color

    def drive(self):            # метод экземпляра
        return f"{self.model} едет по дороге!"
```

1.26. Магические методы (init, str и др.)

Магические методы (Dunder — Double Underscore) позволяют встраивать объекты в стандартный синтаксис Python.

1. Создание и удаление (везде __ текст__)

* __init__ — инициализация (конструктор). * new — создание экземпляра. * del — удаление объекта.

2. Отображение

* str — для людей (print(), str()). * repr — для отладки (вид в консоли).

3. Математика и Сравнение

* add, sub, mul — работа операторов +, -, *. * eq, lt, gt — работа ==, <, >.

4. Контейнеры (как списки/словари)

* len — длина len(obj). * getitem — чтение по индексу obj[i].

* setitem — запись по индексу obj[i] = x. * contains — проверка in.

5. Поведение функции и контекста

* call — позволяет вызывать объект как функцию obj(). * enter, exit — работа с оператором with.

```
class Point:

    def __init__(self, x, y):

        self.x, self.y = x, y

    # Сложение объектов: p1 + p2

    def __add__(self, other):

        return Point(self.x + other.x, self.y + other.y)

    # Печать объекта: print(p)

    def __str__(self):

        return f"({self.x}, {self.y})"

p1 = Point(1, 2)

p2 = Point(3, 4)

print(p1 + p2)  # Выведет: (4, 6)
```

1.27. Объекты класса

Объект — это конкретный экземпляр класса.

1. **Создание**: Происходит путём вызова класса как функции: car = Car().
2. **Доступ к атрибутам**: Через точку: car.model = "Tesla".
3. **Вызов методов**: car.start_engine(). При вызове Python сам подставляет объект в качестве первого аргумента (self).

Пример:

Python

```
my_car = Car("Tesla Model 3", "красный")
print(my_car.model)          # Tesla Model 3
print(my_car.drive())        # Tesla Model 3 едет по дороге!
```

Можно динамически добавлять атрибуты: my_car.year = 2025.

1.28. Декораторы

Декораторы — это «обертки», которые позволяют менять или расширять функционал функции, не переписывая её основной код.

Декоратор принимает функцию как аргумент, создаёт внутри себя новую функцию-обёртку, добавляет логику (например, замер времени или проверку прав) и возвращает эту обёртку.

Синтаксический сахар: @decorator_name над функцией.

Пример:

```
def timer(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(f"Функция выполнилась за {time.time() - start:.4f} сек")
        return result
    return wrapper
@timer
def slow_calculation(n):
    time.sleep(1)
    return n * n

slow_calculation(10)
```

Популярные встроенные декораторы: @property, @classmethod, @staticmethod, @lru_cache.

1.29. Наследование

Наследование позволяет создать класс на основе другого, чтобы избежать дублирования кода.

- **Базовый (родительский) класс**: Содержит общую логику.
- **Производный (дочерний) класс**: Наследует всё от родителя и может добавлять свои методы или переопределять старые.

В Python поддерживается **множественное наследование** (один класс от нескольких родителей). Порядок поиска методов определяется **MRO** (Method Resolution Order) — можно посмотреть через Class.__mro__.

Пример:

Python

```
class Animal:
    def speak(self):
        return "Какой-то звук"
class Dog(Animal):
    def speak(self):        # переопределение
        return "Гав!"
class Cat(Animal):
    def speak(self):
        return "Мяу!"
dog = Dog()
print(dog.speak())          # Гав!
```

Используйте super().__init__() для вызова конструктора родителя.

1.30. Инкапсуляция

Инкапсуляция — это объединение данных и методов в одну «капсулу» и ограничение прямого доступа к ним извне.

- **Публичные** (name): Доступны везде.
- **Защищённые** (_name): Сигнал другим программистам — «это внутреннее, не трогай», но технически доступ есть.
- **Приватные** (__name): Имя искажается (_ClassName__name), затрудняя прямой доступ. Используется для защиты критически важных данных.

```
class SmartHome:

    def __init__(self):

        self.model = "Xiaomi v2"      # Публичный: видно всем (public)

        self._temp = 22               # Защищенный: для внутреннего пользования и наследников
(protected)

        self.__password = "admin"     # Приватный: скрыт даже от наследников (private)
```

1.31. Полиморфизм

Полиморфизм — принцип, позволяющий использовать один и тот же интерфейс для разных типов данных.

Пример: У классов Dog и Cat есть метод speak(). Мы можем пройтись циклом по списку животных и вызвать animal.speak(), не проверяя, кто именно перед нами — каждый «ответит» по-своему (утиная типизация).

Пример:

Python

```
animals = [Dog(), Cat()]
for animal in animals:
    print(animal.speak())   # Гав! / Мяу!
```

1.32. Абстракция

Абстракция — скрытие сложных деталей реализации и выделение только самого важного.

Реализуется через абстрактные классы (from abc import ABC, abstractmethod).

Мы создаём «шаблон», где методы только объявлены, но не написаны. Дочерние классы обязаны реализовать эти методы, иначе объект создать не получится.

Пример:

Python

```
from abc import ABC, abstractmethod


class Shape(ABC):
    @abstractmethod
    def area(self):
        pass


class Circle(Shape):
    def __init__(self, r):
        self.r = r


    def area(self):
        return 3.14 * self.r ** 2
```

1.33. Исключения и их обработка

Исключения — ошибки, возникающие во время работы программы (Runtime errors). Все стандартные ошибки (ValueError, TypeError и др.) — это классы. Можно создавать свои, наследуясь от Exception.

Конструкция:

```
try:
    # опасный код
    x = 1 / 0
except ZeroDivisionError as e:
    print("Деление на ноль!", e)
except Exception as e:
    print("Общая ошибка:", e)
else:
    print("Всё прошло без ошибок")   # выполняется, если исключений не было
finally:
    print("Выполняется всегда")      # очистка ресурсов
```

Своё исключение:

```
class NegativeValueError(Exception):
    pass
if value < 0:
    raise NegativeValueError("Значение не может быть отрицательным")
```

1.34. Параллельное программирование. GIL

В Python есть три способа делать несколько дел одновременно. Выбор зависит от того, во что упирается задача: в процессор (CPU) или в ожидание (I/O).

1. Threading (Потоки) — для сетевых задач

Использует один процесс и общую память. Подходит для задач ожидания: чтение файлов, запросы к API, работа с БД.

* Минус: GIL (Global Interpreter Lock) не дает потокам выполнять Python-код одновременно на разных ядрах.

* Когда: Загрузка 100 сайтов.

2. Multiprocessing (Процессы) — для вычислений

Запускает отдельный экземпляр Python на каждом ядре процессора. У каждого процесса своя память. GIL здесь не мешает.

* Минус: Потребляет много оперативной памяти, сложно обмениваться данными между процессами.

* Когда: Обработка тяжелых фото, видео, сложные математические расчеты.

3. Asyncio (Асинхронность) — для высокой нагрузки

Работает в одном потоке и одном процессе. Не переключает контекст, а «ставит задачу на паузу», пока ждет ответа от сети.

* Плюс: Самый быстрый и экономный способ для работы с тысячами сетевых соединений (чат-боты, веб-серверы).

* Когда: Написание высоконагруженного сервера на FastAPI или aiogram.

1.35. Многопоточное и многопроцессорное программирование

- **Multithreading (threading)**: Несколько потоков в одном процессе. Хорошо для сетевых операций (пока один поток ждёт ответ от сайта, другой работает). Сильно ограничены GIL.
- **Multiprocessing (multiprocessing)**: Создание нескольких процессов. У каждого свой GIL и своя память. Это основной способ реально задействовать все ядра процессора для тяжёлых вычислений.

Пример многопроцессорности:

Python

```
from multiprocessing import Pool
def square(n):
    return n * n
with Pool(processes=4) as pool:
    results = pool.map(square, range(100))
```

1.36. Асинхронное программирование

Асинхронное программирование — способ выполнения задач без блокировки потока. Вместо того чтобы ждать завершения операции (например, загрузки файла), программа переключается на другую задачу.

- **asyncio** — основная библиотека.
- **async def** — объявляет корутину (функцию, которую можно приостановить).
- **await** — указывает место, где можно «подождать», пока выполнится асинхронная операция.

```
import asyncio

async def fetch(url):# асинхронный запрос

        await asyncio.sleep(1)


    return f"Данные с {url}"
async def main():
    tasks = [fetch("url1"), fetch("url2")]
    results = await asyncio.gather(*tasks)
    print(results)
asyncio.run(main())
```

1.37. Тестирование. Назначение. Виды

Тестирование — это проверка того, что код делает именно то, что задумано. **Основные виды:**

- **Unit-тесты**: Проверка минимальных частей кода (одной функции).
- **Интеграционные**: Проверка, как несколько модулей работают вместе.
- **Регрессионные**: Проверка того, что новые изменения не сломали старый функционал.

1.38. Ручное тестирование

Ручное тестирование — процесс, когда человек вручную проходит по приложению, нажимает кнопки и проверяет результат.

Плюсы: Можно заметить визуальные баги или неудобство интерфейса. **Минусы**: Долго, дорого, присутствует человеческий фактор (можно что-то пропустить).

1.39. Автоматизация тестирования

Автоматизация тестирования — написание программ, которые сами тестируют ваше приложение.

Инструменты: pytest (самый популярный), unittest (встроенный), selenium / playwright (для браузеров).

Плюсы: Мгновенная проверка после каждого изменения, высокая надёжность, можно запускать тесты тысячи раз.

```
def add(a, b): return a + b

def test_add():


  assert add(2, 3) == 5
    assert add(-1, 1) == 0
```